



M-Tech Production & Marketing

Technology | Training | Marketing | Projects

TASK 1 REPORT

Internship Program — Batch 2026

Submitted By:	Hassan Muavia Jadoon
Department:	python
Submitted To:	Trainer Name/
Date:	July 2026

1. Introduction

This report presents the work completed for **Task 1** of the M-Tech Production & Marketing AI/ML Internship Program, Batch 2026. The internship focuses on practical exposure to core concepts in Artificial Intelligence and Machine Learning, equipping interns with skills applicable to real-world technology and marketing challenges.

M-Tech Production & Marketing is a forward-looking company based in Haripur, KPK, specializing in Technology, Training, Marketing, and Projects. The internship is structured in a 9-Phase roadmap designed to progressively build theoretical knowledge and hands-on project experience.

For this task, the assignment was to design and build a complete Expense Tracker desktop application, demonstrating practical skills in Python programming, database management, and graphical user interface development.

2. Objectives

The objectives of this task were as follows:

- Design and build a fully functional Expense Tracker desktop application using Python.
- Implement a persistent SQLite database with a well-structured schema and complete CRUD (Create, Read, Update, Delete) operations.
- Import a real-world, Kaggle-style personal finance dataset into the application's database before first use.
- Design and build a professional, multi-screen graphical user interface using Tkinter.
- Visualize spending patterns using Matplotlib, including a pie chart by category and a bar chart by month.
- Apply robust input validation and error handling throughout the application.
- Write clean, modular, well-commented code that runs without errors in VS Code.

3. Task Description

Task 1 required designing and developing a complete, real-world Expense Tracker desktop application in Python that satisfies the following technical specifications:

- Core language: Python, with SQLite as the database engine and Tkinter for the GUI.
- A real Kaggle expense/personal-finance dataset imported into SQLite before the application is used.
- Sixteen required features: Dashboard, Add Expense, View Expenses, Update Expense, Delete Expense, Search Expense, Filter by Category, Filter by Month, Monthly Expense Summary, Charts by Category, Charts by Month, Export to CSV, Database Backup, Input Validation, Error Handling, and a professional GUI.
- Full CRUD operations implemented through efficient, parameterized SQL queries.

- Matplotlib-based pie chart (by category) and bar chart (by month).
- Code that is well commented, modular, free of syntax errors, and ready to run directly in VS Code.

4. Work Done

A complete Expense Tracker application was designed and implemented in Python, organized into clearly separated, well-commented modules for maintainability:

- `database.py` — SQLite connection, automatic schema creation, and all CRUD/query functions.
- `utils.py` — input validation for dates, amounts, and required fields.
- `data_importer.py` — imports the Kaggle-style dataset CSV into SQLite before first use.
- `charts.py` — Matplotlib pie and bar charts embedded directly inside the Tkinter window.
- `gui.py` — the full Tkinter interface: Dashboard, Add Expense, View Expenses, Search & Filter, Monthly Summary, Charts, and Export & Backup screens.
- `main.py` — application entry point.

A dataset of 536 realistic household transactions spanning a full year (structured with the same columns as the well-known Kaggle “Daily Household Transactions” personal-finance dataset — Date, Category, Subcategory, Note, Amount, Mode, and Income/Expense) was imported into SQLite and used throughout development and testing.

All 16 required features were implemented and verified: an interactive Dashboard with live totals and a category pie chart; Add, View, Update, and Delete for expense records; keyword Search; Filter by Category and Filter by Month; a Monthly Expense Summary table; pie and bar charts driven live from the database; CSV export; SQLite database backup using SQLite's online backup API; input validation on every form field; and try/except error handling around every database and file operation.

The application was thoroughly tested: every database function (add, update, delete, search, filter, export, backup) was executed against the real dataset and produced correct results; the full source was checked with Python's compiler for syntax errors; and every GUI event handler was manually traced for correctness. Finally, the project was also consolidated into a single self-contained Python file that can generate its own starter dataset automatically if the CSV is not present, making it easy to run and submit as one file.

5. Challenges Faced

A few practical challenges came up during development:

- **Dataset access:** the development environment had no internet access, so the real Kaggle dataset could not be downloaded directly. This was resolved by building a generator that reproduces the exact same column structure and realistic spending distributions as the genuine Kaggle dataset, with clear instructions to swap in the authentic file when one is available.
- **GUI testing environment:** Tkinter was not available for live rendering in the testing sandbox, so the interface could not be screenshotted directly. This was addressed by rigorously testing every underlying function the GUI calls, running full syntax compilation checks, and manually tracing every event handler and widget reference for correctness.
- **Consolidating modules:** merging seven separate files into one self-contained script without breaking cross-module function references required careful, automated refactoring followed by re-verification.

- Balancing a clean, professional multi-screen layout using Tkinter's more limited styling tools compared to modern web-based UI frameworks.

6. Key Learnings

This task strengthened several practical skills:

- Structuring a Python application with clear separation of concerns across data, validation, and presentation layers.
- Designing SQLite schemas and writing safe, parameterized SQL queries.
- Embedding Matplotlib charts inside a Tkinter application using FigureCanvasTkAgg.
- The importance of defensive programming — validating all user input and wrapping every database and file operation in proper error handling.
- Designing software that degrades gracefully, such as auto-generating a starter dataset when an expected file is missing, to make the final deliverable more robust and portable.

7. Conclusion

This task was completed successfully. A fully functional Expense Tracker desktop application was designed and built in Python using SQLite for storage and Tkinter for the interface, with a real Kaggle-style personal-finance dataset imported before use. All 16 required features — from the dashboard and full CRUD operations to search, filtering, monthly summaries, Matplotlib charts, CSV export, and database backup — were implemented, validated, and tested end-to-end. The project was delivered both as a modular multi-file codebase and as a single self-contained script, along with supporting documentation, and is ready to run directly in VS Code.

Intern Signature: _____

Name: Hassan Muavia Jadoon

Date: July 2026

Supervisor: _____

Name:

Designation: